Module d'audit SAGA : changements d'implémentation

Oct 26, 2026 · @Alphamar

L'audit sort du chemin de la requête et s'appuie sur ce que Spring fournit déjà : mêmes tables, une seule écriture par échange, un seul traceId partout.

Ce qui change et pourquoi

Sujet	Avant	Maintenant	Pourquoi
Organisation	Classes d'audit dans le module web	Module `saga-service-audit`, branché par simple dépendance ; toute la config dans `AuditConfiguration`	Un seul endroit à lire et à faire évoluer
traceId	UUID généré par `MdcFilter` et `currentTraceId()`, envoyé dans `x-b3-trace-id`	traceId Micrometer, renvoyé au client par `TraceIdResponseFilter`	Le même ID dans les logs, l'audit et Zipkin ; plus de header B3 invalide ni de fuite de MDC sur les threads `@Async`
Écriture en base	2 à 4 accès synchrones par requête ; 2 transactions `REQUIRES_NEW` par appel Feign	Copie déposée dans une file, écrite par 2 threads dédiés (`ThreadPoolTaskExecutor`)	Aucune latence ajoutée ; au plus 2 connexions prises au pool
Nombre d'écritures	Insert au début, update à la fin	Une seule ligne par échange	Deux fois moins d'accès base
Feign	`@Primary Client.Default` qui remplaçait le client Spring Cloud	`Capability` qui enveloppe le client existant	On garde load balancer, pool de connexions et SSL
Réponse Feign	Lue entièrement en mémoire	Seul le début est lu, le reste est rendu intact	Une grosse réponse ne sature pas la mémoire
Consultation / suppression	Entité et id devinés par position dans l'URL	Annotations `@ConsultationLog` / `@DeletionLog` lues par un `HandlerInterceptor`	`/persons/42/addresses/7` donne bien l'adresse 7
Requêtes en erreur	Non auditées (pas de `try/finally`)	Auditées avec leur statut	Ce sont justement celles qu'on veut tracer
Masquage	Regex sur les valeurs texte ; seul `Authorization` masqué	Jackson à toute profondeur et pour tout type ; `Cookie` et `Authorization` masqués	Un `"pin": 1234` ou un cookie de session ne fuit plus
Body	`CachedServletRequest` sans limite	`ContentCachingRequestWrapper` de Spring, limité à `REQUEST_BODY_MAX_SIZE`	Moins de code maison, mémoire sous contrôle

Ce qu'on garde

- Les tables `AUDIT_ENTRY` et `AUDIT_OUTGOING`, plus deux colonnes sur `AUDIT_ENTRY` : `URI_TEMPLATE` et `PATH_VARIABLES`.

- L'application appelante : header `x-sender`, sinon `azp` du token.
- `UNKNOWN_USER` quand personne n'est authentifié.
- La troncature avec le marqueur `...[truncated]`, basée sur `ValidationConstraints`.
- Le principe : l'audit ne fait jamais échouer une requête.

Le compromis assumé

L'audit technique peut perdre un événement si la file est pleine ou si l'application plante. Un warning est loggé dans le premier cas. Les journaux de consultation et de suppression, eux, restent écrits de façon synchrone.

Réglages à retenir

- `audit.writer-threads` : 2 par défaut, au plus 1/4 du pool Hikari.
- `audit.queue-capacity` : 10 000 événements en attente au maximum.
- `audit.included-paths` / `audit.excluded-paths` : chemins audités, `/actuator/**` exclu.